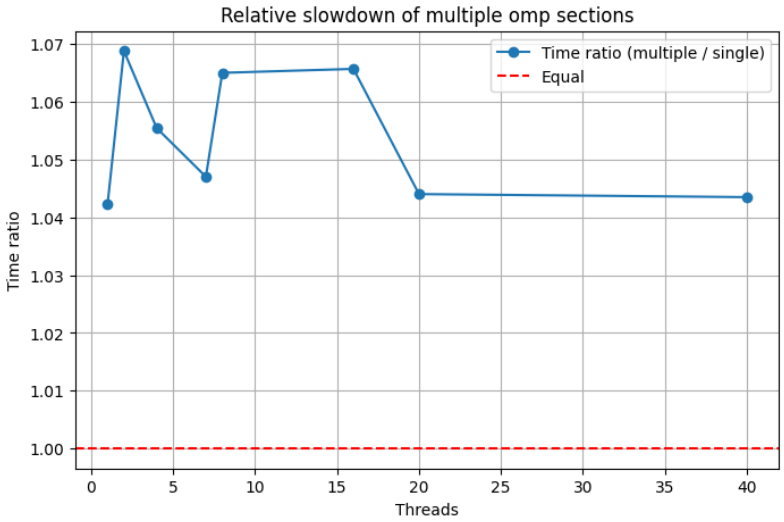
Задание 3

Вычислительный узел

Наименование:	ProLiant XL270d Gen10
CPU	
Model name:	Intel(R) Xeon(R) Gold 6248 CPU @ 2.50GHz
Thread(s) per core:	2
Core(s) per socket:	20
Socket(s):	2
CPU max MHz:	3900.0000
CPU min MHz:	1000.0000
Numa-nodes	
available:	2 nodes
node 0 size:	385636 MB
node 1 size:	387008 MB
OS	
Name:	Ubuntu
Version:	22.04

Анализ различных реализаций алгоритма

Сравнение реализации алгоритма с одной и множеством omp-секций

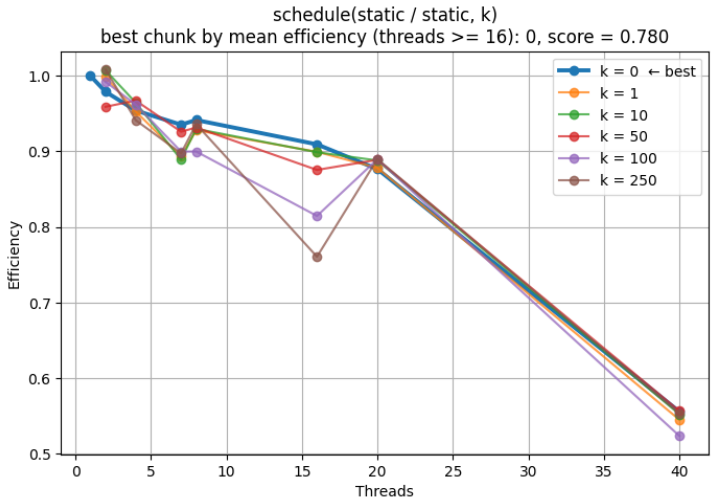


По графику видно, что реализация с несколькими параллельными секциями во всех случаях работает медленнее, чем вариант с одной параллельной областью. Относительное замедление составляет примерно 4–7% и сохраняется практически на всём диапазоне числа потоков.

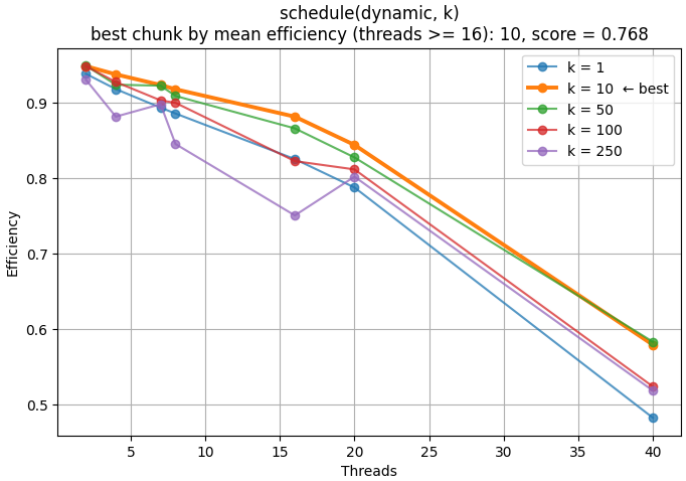
Основной причиной является наличие дополнительных накладных расходов: создание и завершение параллельных регионов, синхронизация потоков между ними, а также ухудшение локальности данных. В варианте с одной параллельной областью потоки создаются один раз и используются повторно, что снижает накладные расходы.

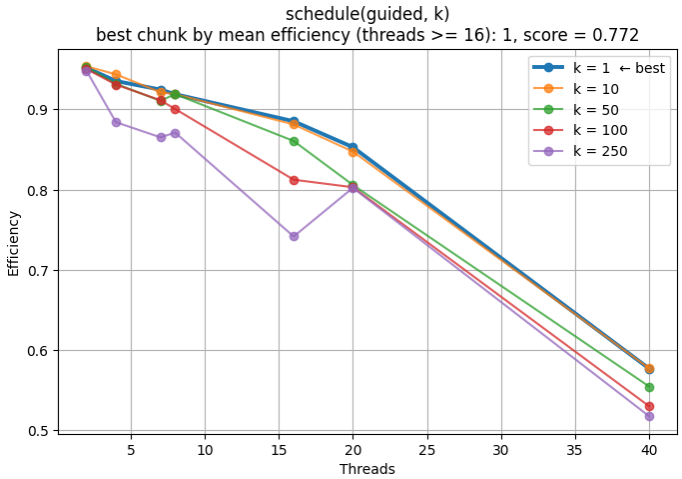
Таким образом, использование нескольких параллельных секций в данном алгоритме оказывается неэффективным, и предпочтительным является вариант с одной параллельной областью. В дальнейших тестах будет использоваться именно такая реализация.

Подбираем лучший scheduler и chunk size (k)



* здесь под k = 0 понимается schedule(static)





По представленным графикам эффективности для различных стратегий планирования можно сделать несколько выводов. Во всех случаях с увеличением числа потоков наблюдается постепенное снижение эффективности, что связано с ростом накладных расходов и ухудшением масштабируемости при высоком уровне параллелизма.

Для schedule(static) наилучшие результаты достигаются при отсутствии разбиения на чанки (k = 0), что соответствует стандартному статическому распределению. Это объясняется минимальными накладными расходами и хорошей локальностью данных.

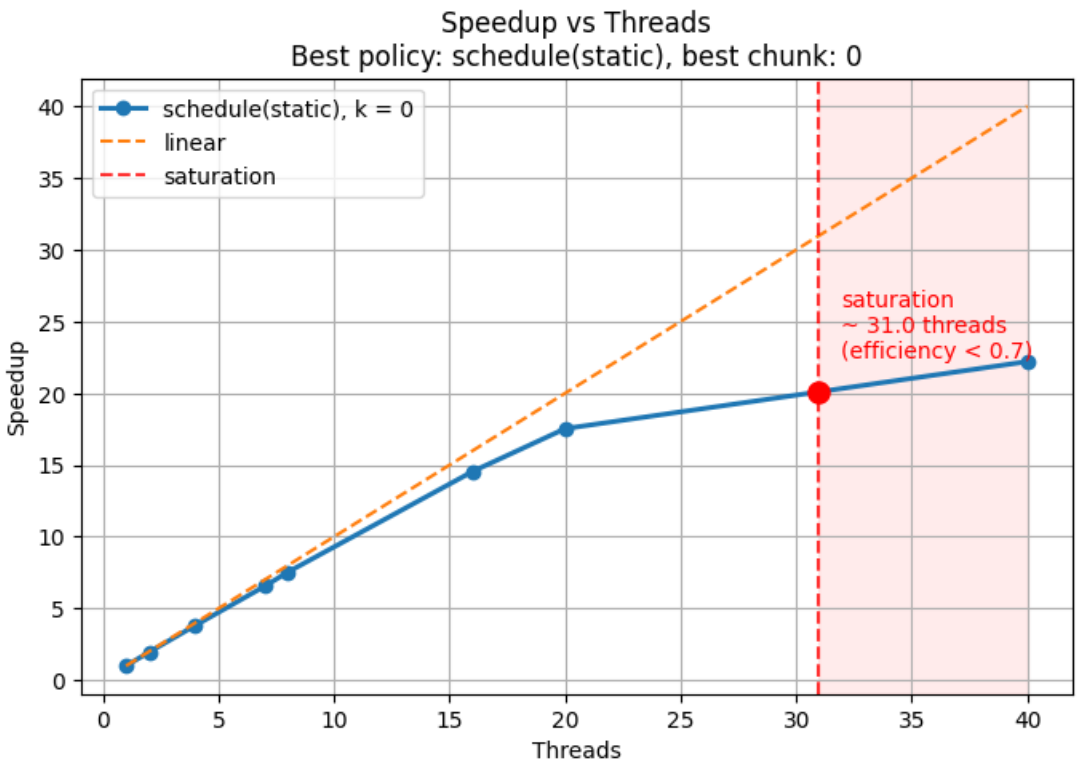
Для schedule(dynamic) оптимальным оказался промежуточный размер чанка (k = 10), поскольку слишком маленькие чанки увеличивают накладные расходы на планирование, а слишком большие ухудшают балансировку нагрузки.

Для schedule(guided) наилучший результат достигается при минимальном размере чанка (k = 1), что позволяет эффективно сочетать динамическое распределение нагрузки с уменьшением накладных расходов на поздних этапах выполнения.

Сравним эти стратегии между собой. Для метрики сравнения выберем усреднённую эффективность на большом количестве потоков (16 и выше):

№	schedule	chunk size	mean efficiency
1	static	0	0.780
2	guided	1	0.772
3	dynamic	10	0.768

В целом, различия между стратегиями планирования оказываются сравнительно небольшими, однако статическое распределение демонстрирует немного более высокую эффективность на больших числах потоков, что указывает на его лучшую масштабируемость для данной задачи.



Вывод

По графику ускорения для лучшей конфигурации (schedule(static)) видно, что алгоритм демонстрирует близкое к линейному масштабирование до примерно 30 потоков. После этого наблюдается точка насыщения (~ 31 поток), при которой эффективность падает ниже 0.7, и дальнейшее увеличение числа потоков даёт лишь незначительный прирост производительности.

Это указывает на то, что после достижения данного уровня параллелизма накладные расходы (синхронизация, конкуренция за ресурсы, влияние NUMA) начинают доминировать, и добавление новых потоков становится нецелесообразным.